

# Daily Scholar Papers Report — 2026-08-19

---

2026-08-19

## Daily Scholar Papers Report — 2026-08-19

---

[Download PDF](#)

**Window covered:** 2026-08-18 → 2026-08-19 (Google Scholar alerts + user-curated self-emails, last 24 h)

---

### Executive Summary

---

No new papers arrived in today's window, and the more interesting fact is *how nearly this report failed to notice*.

The primary Scholar query returned one thread. One thread is a normal result — most days in this archive return between one and four. But the thread was the same one processed yesterday, delivered at 02:17 UTC on 18 August, containing the same six candidates whose triage and three full deep-reads make up the 2026-08-18 report. Nothing in it was new. Had the thread been taken at face value, today's report would have been a near-verbatim reproduction of yesterday's, published under a new date, and the duplication would have been invisible from inside either document.

The cause is a property of the query rather than a fault anywhere. Gmail's `newer_than:1d` is not a rolling twenty-four-hour interval measured from the instant of the query; it is coarser than that, and a thread delivered early on one day still satisfies it when asked again the following day. So on any day where the previous run's newest thread is under roughly two calendar days old and nothing new has arrived, the query re-serves the previous run's input — indistinguishable, in the response itself, from a genuine delivery. **The retrieval window is not a de-duplicator, and a daily pipeline that treats it as one will eventually republish its own output.** Detection today was incidental: the thread identifier was recognised while checking the prior report's formatting. That is a near-miss, not a control, and it has been logged as the run's top carry-forward action — the pipeline should keep a record of thread identifiers it has already consumed and subtract them before screening, rather than relying on a date filter to be exact about something it was never exact about.

With the duplicate removed, the window is empty. The self-email queue also returned nothing, at both the standard filter and a three-day widened one.

Three controls establish that the emptiness is real and that it is unremarkable. The primary query was restated in absolute-date form with spam and trash included, and returned the same single stale thread — ruling out both a relative-date parsing error and a newer alert hidden by misclassification. The self-email query was re-run with its subject filter dropped and its window tripled, and still returned nothing. Most usefully, the Scholar channel was probed at a seven-day radius: **thirteen threads**, arriving in four distinct batches on 13, 14, 15, 16 and 18 August. The feed is healthy and delivering every one to two days; today is an ordinary one-day gap in a working channel. This is worth stating plainly because this archive has recorded the other kind — the six consecutive empty days at the start of August, which

turned out to indicate an upstream subscription problem. The distinction between the two is exactly what the liveness probe exists to draw, and today it comes down on the benign side. No escalation is warranted.

One further item of substance: **yesterday's report was completed and published today**. The 2026-08-18 run wrote both reports to disk but stopped before rendering, validation and commit, leaving a finished analysis of three strong papers unpublished and the site a day stale. Those steps ran today — feedback buttons injected into all four cards, both PDFs rendered, the site rebuilt under `--strict`, and the report committed alongside this one. Readers arriving for the 18 August analysis of **CVE-Genie**, **ConfFuzz** and **SyzMini** will find it live now.

**Outstanding:** 0 · **Keep:** 0 · **Borderline High-Priority:** 0

## Highlighted Papers

---

No new papers in this window. The most recent analysis is the [2026-08-18 report](#), published today, covering CVE-Genie (CCS 2026), ConfFuzz (ASE 2026) and SyzMini (ACM TOCS 2026 / USENIX ATC 2025).

---

## Cross-Paper Synthesis

---

Not applicable — empty window.

## Writing & Rationale Insights

---

Two things worth keeping from a run with no papers in it.

**A retrieval filter is not an identity check, and conflating the two is a general bug.** The failure available today has a shape that recurs well beyond mail queries: a system selects records by *when they arrived*, then treats the selection as though it meant *what it has not yet seen*. Those coincide only when the window boundary is exact and the schedule never drifts. Neither holds here — the window is coarse by design, and a run that fires a few hours late widens it further. The same pattern turns up wherever a pipeline paginates by timestamp, polls a feed by modification date, or resumes a job from a watermark: the filter narrows the candidate set, but only a durable record of what was actually consumed can establish novelty. The fix is unglamorous and cheap — store the identifiers, subtract them — and its value is that it converts a property the system was *hoping* held into one it *checks*. The reason to write this down rather than quietly patch it is that the near-miss is the informative part. The duplicate was caught by recognising a thread ID while looking at something else, which is to say it was caught by luck, and a control that only works when someone happens to be paying attention is not a control.

**Defensive formatting against a script you cannot run is worth the effort, and today proved it.**

Yesterday's run could not execute the feedback-button injector. Rather than hand-writing a dozen percent-encoded issue URLs — unverifiable without running anything, and silently corrupting downstream data if malformed — it hand-formatted the four card headers to match the script's expected shape precisely and left the injection undone. Today the script ran and matched all four cards on the first pass, with no manual repair. The general principle: when a build step is unavailable, the useful move is not to simulate its *output*, which you cannot validate, but to guarantee its *input contract*, which you can.

The first is a guess that fails silently; the second is a promise that fails loudly. The same run also omitted a download link rather than pointing it at a file that did not exist yet, on the same reasoning — a strict build catches the dangling link, so the honest gap is cheaper than the optimistic placeholder. Both choices cost a little completeness on the day and saved the work entirely.